FlowGuard 5000
Pump Health Monitoring System

ENS 184 Mini-Project Manual
Version 2

Department of Engineering

May 1, 2026

Contents

1 Project Background

1.1 Industrial Context: Why Pump Health Monitoring Matters

Centrifugal pumps are one of the most widely used pieces of rotating machinery in industry — they move fluids in water treatment plants, oil refineries, chemical processing facilities, and power stations. When a pump fails unexpectedly the consequences can range from costly downtime to safety hazards. Traditional maintenance approaches are either *reactive* (fix it after it breaks) or *preventive* (replace parts on a fixed schedule regardless of condition). Both approaches waste money: reactive maintenance is expensive and disruptive, while preventive maintenance replaces healthy components prematurely.

Predictive Maintenance (PdM) is a modern engineering strategy that uses sensor data and mathematical models to estimate the *remaining useful life* (RUL) of a component before it reaches failure. Rather than reacting to a breakdown or following an arbitrary calendar schedule, maintenance crews can be dispatched at the optimal time — just before the machine would fail — minimising cost and maximising uptime. This project gives you hands-on experience with the core numerical methods that power real PdM systems.

1.2 The FlowGuard 5000 System

The **FlowGuard 5000** is an industrial centrifugal pump used to move fluid around a closed-loop process system. It is instrumented with two sensors:

- A **voltage sensor** attached to the pump's pressure transmitter. The transmitter outputs a voltage signal (mV) whose relationship to actual differential pressure (bar) must be established by *calibration*. This relationship is not perfectly linear — it depends on the transmitter's electronics — so a curve-fitting approach is required.
- A **vibration sensor** mounted on the pump's bearing housing. Vibration amplitude ($\mathrm{mm/s_{rms}}$) is one of the most reliable indicators of bearing health: as a bearing degrades, the vibration it produces grows steadily over time. When vibration exceeds a manufacturer-specified *failure threshold*, the bearing must be replaced immediately to avoid catastrophic failure.

Historical data from 120 operational test points has been collected and stored in the file `pump_health_data.csv`. Your team will analyse this dataset to build the numerical engine of a health monitoring system.

1.3 The Engineering Problem Your Team Must Solve

Mission Statement

Given 120 historical sensor readings from the FlowGuard 5000, your team must address the following engineering questions:

1. **How does the pressure sensor behave?** The voltage-to-pressure relationship must be modelled mathematically so that future voltage readings can be converted to meaningful pressure values.
2. **How is the bearing degrading?** The vibration data must be modelled as a function of operational hours so that the degradation trend can be characterised.
3. **When will the bearing fail?** Using the degradation model, predict the operational hour at which the vibration will first cross the failure threshold — this is the *predicted remaining useful life*.

4. **How healthy is the pump right now and into the future?** Translate the vibration trend into a normalised Health Index (HI) and produce a dashboard report that a maintenance engineer can act on.

Your answers to these four questions must be grounded in quantitative evidence — numerical models fitted to data, supported by accuracy metrics, and communicated through well-labelled figures.

1.4 System Data Pipeline

The four student modules are not independent — they form a sequential data pipeline. Understanding the order in which data flows through your system is essential before you begin implementing:

Stage	Owner	What happens here
1	Student D	Raw CSV is read from disk and parsed into numeric column vectors (`pressure`, `voltage`, `hours`, `vibration`).
2	Student A	A calibration model is fitted to the (`voltage`, `pressure`) pairs. This model can convert any future voltage reading into an estimated pressure.
3	Student B	A degradation model is fitted to the (`hours`, `vibration`) pairs. This model describes how vibration grows with operating time.
4	Student C	The accuracy of both models (A and B) is quantified. The degradation model is then used to predict when the pump will reach the failure threshold.
5	Student D	The vibration data and model are used to compute a Health Index and generate a dashboard figure. `main.sce` orchestrates all stages in the correct order.

Notice that Stage 5 depends on Stage 3, and Stage 4 depends on Stages 2 and 3. **Integration** — getting all four modules to work together correctly — is as important as making each module correct in isolation.

1.5 Sensor Data

The file `pump_health_data.csv` contains 120 rows of sensor readings with four columns:

Column	Variable	Units	Description
1	`pressure`	bar	Differential pressure across pump
2	`voltage`	mV	Sensor voltage output
3	`hours`	h	Cumulative operational hours
4	`vibration`	$\mathrm{mm/s_{rms}}$	Vibration amplitude

The first line of the file is a header row starting with `#` and must be skipped during parsing. The **failure threshold** is $\xi_{\mathrm{th}} = 9.5\ \mathrm{mm/s_{rms}}$.

1.6 Team Structure

Student	Primary module file	Functions to implement
A	`interpolation/interpolation.sci`	`poly_fit`, `eval_poly`
B	`differentiation/differentiation.sci`	`linearize_vib`, `predict_vibration`
C	`integration/integration.sci`	`goodness_of_fit`, `find_threshold_hour`
D	`data_loader/load_data.sci`, `user_interface/health_dashboard.sci`	`load_data`, `compute_health_index`, `plot_report`, `main.sce`

2 Repository Structure

```
pump_health_data.csv                <- sensor data (provided, at project root)
phm_project/
  main.sce                          <- master script (Student D completes)
  data_loader/
    load_data.sci                   <- Student D, Task D1
  interpolation/
    interpolation.sci               <- Student A
  differentiation/
    differentiation.sci             <- Student B
  integration/
    integration.sci                 <- Student C
  user_interface/
    health_dashboard.sci            <- Student D, Task D2
  test_suites/
    run_all_tests.sci               <- run to test all modules
    test_studentA.sci
    test_studentB.sci
    test_studentC.sci
    test_studentD.sci
    test_integration.sci
```

3 Getting Started with Scilab

Note to Students

Scilab is the *tool* your team uses to implement the numerical algorithms. The intellectual challenge of this project is **numerical methods** — least-squares regression, exponential curve fitting, bisection root-finding — topics you have already studied in class. Scilab is free and open-source with a syntax very close to MATLAB. If you are new to it, this section and the Quick Reference (Appendix A) give you everything you need. **Run `warmup_scilab.sce` first** before opening your own skeleton file — it will build confidence and catch any environment issues.

3.1 Installing and Running Scilab

Download Scilab from `https://www.scilab.org` (free, all platforms). Once installed, open Scilab and use the *File Browser* panel to navigate to `phm_project/`, or type directly in the console:

```
cd '/full/path/to/Version2/phm_project';  // set working directory
exec('warmup_scilab.sce', -1);            // run the warm-up tutorial
exec('main.sce', -1);                     // run the full project
```

The `-1` flag tells Scilab not to echo each line as it executes. Use `exec()` to load function definitions from `.sci` files into memory — not `run` or `source`.

3.2 Three Rules That Differ from Most Languages

1. **Indexing starts at 1, not 0.** The first element of vector `v` is `v(1)`. The last element is `v($)` (the dollar sign means "last index").
2. **Booleans are `%t`/`%f`; infinity is `%inf`.** Test with `isinf(x)` rather than `x == %inf`, because floating-point equality with infinity is unreliable.
3. **A semicolon suppresses console output.** Inside functions and loops, end every assignment with a semicolon (`a = 3;`) to keep the console readable. Omit it temporarily (`a = 3`) to inspect a value while debugging.

3.3 Mathematics to Scilab: Translation Table

Table 1 maps the mathematical notation used in this manual directly to the corresponding Scilab expression. Consult it whenever you are unsure how to express a formula in code.

Table 1: Mathematical notation → Scilab code quick-reference.

Mathematical expression	Scilab code	Notes
$\mathbf{Z}^T$	`Z'`	Transpose (works for real matrices).
$\mathbf{Ab}$	`A * b`	Matrix–vector multiply.
$(\mathbf{Z}^T\mathbf{Z})\mathbf{c} = \mathbf{Z}^T\mathbf{y}$	`(Z'*Z) \ (Z'*y)`	Normal Equations: solve for $\mathbf{c}$ via backslash.
$a_i \cdot b_i$ (element-wise)	`a .* b`	The leading dot makes multiplication element-wise.
x_i^k (element-wise)	`x .^ k`	Use `.^` on vectors, not bare `^`.
$\mathbf{Z}_{:,j} = \mathbf{x}^{j-1}$	`Z(:,j) = x .^ (j-1)`	Fill design matrix column by column inside a loop.
$\ln(x)$	`log(x)`	Natural log; x must be strictly positive.
e^x	`exp(x)`	Works element-wise on vectors.
$\bar{y} = \frac{1}{n}\sum y_i$	`mean(y)`	Built-in mean function.
n = number of data points	`n = length(x)`	Also `size(x,1)` for the row count.
$\mathbf{0}_{n\times 1}$	`zeros(n, 1)`	Pre-allocate an output column vector.
$\mathbf{1}_{n\times 1}$	`ones(n, 1)`	Useful for horizontal threshold lines in plots.
$(h_L + h_R)/2$	`(h_lo + h_hi) / 2`	Bisection midpoint.

3.4 The Backslash Operator — Your Most Important Tool

The expression `A \ b` solves the linear system $\mathbf{Ac} = \mathbf{b}$ for $\mathbf{c}$ without computing any matrix inverse. When `A` is a *tall* matrix (more rows than columns) it automatically returns the **least-squares** solution.

```
// Least-squares fit via the Normal Equations:
c = (Z' * Z) \ (Z' * y);   // c is the coefficient vector

// Square system (exact solution):
sol = A \ b;
```

Do Not Use `inv()`

`c = inv(Z'*Z) * (Z'*y)` is mathematically equivalent but slower and numerically unreliable — it amplifies rounding errors. It is also forbidden in this project. Always use `\`.

3.5 Warm-Up Tutorial Script

The file `warmup_scilab.sce` in `phm_project/` walks through all the Scilab idioms you will need: variables, column vectors, design matrices, the backslash operator, `log`/`exp`, Horner's method, and a simple plot — using toy examples unrelated to pump data. Each section prints labelled output so you can verify you understand every operation before writing your own functions. The script takes approximately 20 minutes to run through completely.

```
exec('warmup_scilab.sce', -1);
```

4 Theoretical Background

How to Use This Section

This section gives you the essential mathematical concepts behind the four modules your team will implement. Read it alongside your lecture notes. Its purpose is to build your intuition for *why* the methods work and what good results look like — not to tell you how to code each step. When you read a concept here and think "I see how to apply this in my function," you are on the right track.

4.1 Polynomial Regression and Sensor Calibration

A sensor calibration establishes the mathematical relationship between a raw instrument signal and the physical quantity it represents. For the FlowGuard 5000, the voltage transmitter's output does not grow linearly with pressure — the electronics introduce a mild nonlinearity. A polynomial model captures this behaviour with enough flexibility.

In general, a degree-k polynomial relates a predictor variable x to a response y:

$$y = c_0 + c_1 x + c_2 x^2 + \cdots + c_k x^k.$$

When you have n measured data pairs $(x_1, y_1), \ldots, (x_n, y_n)$ with $n \gg k+1$, you cannot find a polynomial that passes exactly through every point (the system would be overdetermined). Instead you seek the coefficient vector $\mathbf{c}$ that *minimises* the total squared prediction error — this is the **least-squares** solution.

The structure that lets you write all n equations compactly as a single matrix equation $\mathbf{Zc} = \mathbf{y}$ is called the **design matrix Z**: it has n rows (one per measurement) and $k+1$ columns, where column j contains the predictor values raised to the power $j-1$. Understanding the shape and content of this matrix is the key starting point for implementing `poly_fit`.

Evaluating the polynomial efficiently. Once you have the coefficients, you need to evaluate the polynomial at many query points. The straightforward approach — computing each power of x and summing — involves redundant multiplications. **Horner's method** reformulates the same computation with far fewer operations by exploiting a nested factorisation of the polynomial. Look this method up and understand the factorisation before implementing `eval_poly`.

Questions to guide your thinking:

- What would happen if you tried to solve the system $\mathbf{Zc} = \mathbf{y}$ directly when $n > k + 1$? Why is there generally no exact solution?
- What does minimising $\sum_i (y_i - \hat{y}_i)^2$ mean geometrically?
- Horner's method factors the polynomial from the *outside in*. Work through an example by hand for $c_0 + c_1 x + c_2 x^2$ before writing any code.

4.2 Exponential Growth and Bearing Degradation

Mechanical fatigue in a rolling-element bearing does not progress at a steady linear rate. As microscopic cracks in the race surface accumulate and propagate, the energy released as vibration grows in a way that is better described by an **exponential** function of time:

$$\xi(h) = a\, e^{bh},$$

where ξ is the vibration amplitude, h is the accumulated operating hours, and a, b are positive constants. The parameter a represents the vibration level at the very beginning of operation (low wear state), while b controls how rapidly the degradation accelerates.

An exponential function is inherently nonlinear in its parameters, which presents a challenge: the least-squares machinery you used for polynomials requires a *linear* relationship between the parameters and the response. A classical engineering technique transforms the exponential model into an equivalent linear form that can be fitted with exactly the same tools. Think carefully about what mathematical operation applied to $\xi(h) = a\, e^{bh}$ produces a linear relationship in h — and what that implies for the form of your design matrix.

Questions to guide your thinking:

- After applying the linearising transformation, which of the two parameters (a or b) appears directly in the linear model, and which must be recovered by an inverse transformation?
- The transformation requires the vibration values to satisfy a particular mathematical condition. What is that condition and why is it physically reasonable for a bearing that is degrading but not yet broken?
- If you plot ξ vs h the curve bends upward. What shape does the same data have after your linearising transformation?

4.3 Model Accuracy: RMSE and R^2

Fitting a model to data is only useful if you can assess how well the model actually describes the data. Two complementary metrics are standard in engineering:

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}, \qquad R^2 = 1 - \frac{\displaystyle\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\displaystyle\sum_{i=1}^{n}(y_i - \bar{y})^2},$$

where y_i are the measured values, $\hat{y}_i$ are the model predictions, and $\bar{y}$ is the sample mean.

Interpreting the metrics:

- RMSE is expressed in the same physical units as the response variable. A smaller RMSE means predictions are closer to the measurements on average.
- R^2 is dimensionless and lies between $-\infty$ and 1 in general. An R^2 of 1 means the model explains all of the variation in the data; $R^2 = 0$ means the model does no better than simply predicting the sample mean; $R^2 < 0$ means the model is worse than the mean predictor.
- A high R^2 with a small RMSE gives confidence that a model is useful. For this project both models should achieve $R^2 > 0.95$ on the provided dataset if implemented correctly.

Edge case: What should R^2 return when all n measurements are identical (i.e. the denominator is zero)? Your implementation must handle this gracefully.

4.4 Finding the Failure Hour by Root-Finding

The degradation model predicts vibration as a continuous, smoothly increasing function of time. At some point that function will cross the failure threshold ξ_{th}. The operational hour at which the crossing occurs is the quantity of interest for maintenance planning — it is the *predicted remaining useful life*.

Mathematically, you are looking for a value h^* such that $\hat{\xi}(h^*) = \xi_{\text{th}}$, or equivalently the root of the function $g(h) = \hat{\xi}(h) - \xi_{\text{th}} = 0$. If you can identify an interval $[h_L, h_R]$ in which g changes sign, the **Intermediate Value Theorem** guarantees that a root exists inside the interval (provided g is continuous). An interval with this property is called a *bracket*.

The **bisection method** is a simple, reliable algorithm for narrowing a bracket to an arbitrarily small interval around the root. Each step cuts the bracket in half based on the sign of g evaluated at the midpoint. The method converges slowly compared to Newton's method, but it *always* converges as long as the initial bracket is valid — which makes it an excellent choice when robustness matters more than speed.

Questions to guide your thinking:

- How can you scan through the fitted vibration vector to locate a consecutive pair of points that bracket the threshold crossing?
- At each bisection step, you evaluate g at the midpoint. Based on the sign of this evaluation, how do you update the bracket for the next step?
- How do you decide when to stop? Think about how narrow the bracket needs to be for a physically useful maintenance prediction.
- What should the function return if the fitted vibration never reaches the threshold within the available data range?

4.5 Health Index and Visual Communication

The Health Index (HI) is a normalised scalar that expresses pump condition on a scale from 0 (perfectly healthy) to 1 (reached failure threshold). It translates a physical measurement (vibration in mm/s) into a unitless score that a maintenance engineer can interpret at a glance, regardless of the specific sensor type or failure threshold.

Good engineering dashboards combine multiple information layers into a single readable figure. For this project the report figure should allow a viewer to answer three questions without reading the caption: (1) How does the vibration trend compare with the fitted model? (2) Has the pump entered a warning or critical zone? (3) When is the predicted failure, and how much time remains?

Questions to guide your thinking:

- Given a vibration reading ξ and the threshold ξ_{th}, write a simple formula for HI that maps 0 (healthy) to 1 (threshold). What happens when $\xi > \xi_{\text{th}}$, and how should HI behave?
- What visual elements (curve, scatter, zone line, marker) are needed in each panel to answer the three questions above?
- Scilab's plotting functions work with vectors, not scalars. If you want a horizontal line at height c, how do you construct the corresponding y-data vector?

Where to Read More

For deeper background, consult:

- Course lecture notes: numerical linear algebra (least-squares), root-finding (bisec-

tion), and data visualisation.
- Golub & Van Loan, *Matrix Computations* — least-squares and numerical conditioning.
- Any introductory numerical analysis text for bisection convergence theory.
- ISO 10816 and ISO 20816 for industrial vibration severity criteria.

5 Student A — Polynomial Calibration

Student A — `interpolation/interpolation.sci`

Your role in the system. Before any analysis can be performed on the pump's pressure data, the raw voltage signal from the pressure transmitter must be converted into pressure values in bar. This conversion is your responsibility. Every pressure value the rest of the team uses ultimately comes from your calibration model.
The relationship between voltage and pressure is not perfectly linear. Laboratory tests show that a **cubic** ($k = 3$) polynomial describes the curve well over the operating voltage range. Your implementation must be general enough to handle any polynomial degree k, so that the same code can be reused for other sensors in the future.

Task A1: `poly_fit(x, y, k)` — Polynomial Least-Squares Fit
- **Input:** predictor vector `x` ($n \times 1$), response vector `y` ($n \times 1$), polynomial degree `k`
- **Output:** `coeff` — coefficient vector of length $k + 1$, ordered from the constant term (c_0) up to the highest power (c_k)

Guiding questions:
- How can you write all n equations of the form $y_i = c_0 + c_1 x_i + \cdots + c_k x_i^k$ as a single matrix equation $\mathbf{Zc} = \mathbf{y}$? What does each column of $\mathbf{Z}$ represent?
- When $n > k+1$ the system is overdetermined — there is generally no exact solution. What does it mean to find the *least-squares* solution instead?
- Scilab provides an efficient, numerically stable way to solve an overdetermined linear system without forming any matrix inverse. What is it, and why should you prefer it over computing $(\mathbf{Z}^T\mathbf{Z})^{-1}\mathbf{Z}^T\mathbf{y}$ explicitly?

Task A2: `eval_poly(coeff, x_query)` — Polynomial Evaluation
- **Input:** `coeff` ($k + 1$ elements from `poly_fit`), `x_query` (scalar or column vector)
- **Output:** `y_query` — polynomial values at each query point (same size as `x_query`)

Guiding questions:
- The naive approach — computing $x^0, x^1, x^2, \ldots, x^k$ and summing — involves many redundant multiplications. Look up **Horner's method** and understand the nested factorisation it exploits. Work through an example by hand for a cubic polynomial before writing any code.
- What order should you process the coefficients? Starting from the constant term or the highest-degree term?
- How do you ensure the function works for both a single query point (scalar) and a vector of query points without changing your code?

Expected Outcomes — How to Verify Your Work

- **Shape:** `coeff` must be a column vector of length exactly $k + 1 = 4$ when called with degree $k = 3$.
- **Direction:** For the voltage→pressure calibration the fitted polynomial should be generally increasing over the sensor range.
- **Fit quality:** When you evaluate the polynomial at the same voltage values used to fit it, the predictions should track the measured pressures closely. Student C's `goodness_of_fit` should report $R^2 > 0.95$.
- **Sanity test:** Try fitting a simple straight line through perfectly linear data and verify the returned coefficients match what you would compute by hand.
- **Vector output:** `eval_poly(coeff, linspace(1,8,50)')` must return a 50-element column vector.

Common Pitfalls

- Row vs. column vectors can cause silent dimension mismatches. Force all inputs to column vectors at the start of each function.
- Using `inv()` to invert a matrix explicitly is numerically unreliable and is not permitted. Use Scilab's built-in linear solver.
- Double-check that your Horner loop processes the coefficients in the correct order and produces the same value as the direct summation on a small example you can verify by hand.

6 Student B — Exponential Vibration Model

Student B — `differentiation/differentiation.sci`

Your role in the system. Your module is the heart of the prognostic engine. By fitting a model to the vibration history, you provide the smooth degradation curve that Students C and D will use to predict the failure hour and compute the Health Index. If your model is inaccurate, every downstream result is compromised.

Bearing degradation does not progress at a steady rate. As microcracks in the bearing race propagate, the rate of energy dissipated as vibration *accelerates*. An exponential function captures this behaviour better than a straight line, but it introduces a challenge: the model $\xi(h) = a\,e^{bh}$ is nonlinear in its parameters a and b. Your task is to find a transformation that makes the fitting problem tractable using the same linear algebra tools that Student A uses.

Task B1: `linearize_vib(hours, vibration)` — Exponential Fit

- **Input:** `hours` ($n \times 1$, cumulative operating hours), `vibration` ($n \times 1$, all values must be strictly positive)
- **Output:** scalars `a` (scale factor) and `b` (growth rate), both physically expected to be positive

Guiding questions:

- The model $\xi = ae^{bh}$ is not linear. Apply a mathematical transformation to both sides so that the result is a linear equation in h. What does the transformed equation look like?
- Once you have a linear equation in h, identify the two unknown quantities that must be estimated. How many columns does your design matrix need?

- The fitting process gives you estimates of the unknowns in the *transformed* space. How do you convert those back to the physically meaningful parameters a and b of the original exponential model?
- What mathematical condition must all vibration values satisfy for the transformation to be valid? Why is this condition physically reasonable for a bearing in service?

Task B2: `predict_vibration(a, b, h_query)` — Exponential Prediction

- **Input:** scale factor `a`, growth rate `b`, query hours `h_query` (scalar or vector)
- **Output:** `vib_query` — predicted vibration amplitude at each query hour

Guiding questions:

- Given a and b, write the formula for $\hat{\xi}(h)$ in closed form.
- How do you apply this formula to a vector of query hours in a single Scilab expression without using a loop?
- Check: at $h = 0$, what should your prediction equal?

Expected Outcomes — How to Verify Your Work

- **Sign:** Both `a` and `b` must be positive. Negative values mean the transformation or the recovery step went wrong.
- **Physical range:** `a` represents the vibration level at the very start of operation (minimal wear), so expect a small positive number in the range 1–5 mm/s. The growth rate `b` should be a small positive number (the vibration roughly doubles over thousands of hours, not hundreds).
- **Fit quality:** Evaluate your model at the same hours used to fit it and compare with the raw vibration data. Student C's `goodness_of_fit` should report $R^2 > 0.95$.
- **Zero-hour check:** `predict_vibration(a, b, 0)` must return exactly `a`.
- **Monotone:** The predicted vibration must be strictly increasing over the operational hour range.

Common Pitfalls

- Scilab's `log` function computes the natural logarithm, which is what the linearisation requires. Verify which base is being used if results look unexpected.
- The transformation is only defined for strictly positive vibration. Check the input data for zero or negative entries and call `error()` with a descriptive message if any are found.
- The parameters a and b come from different parts of the least-squares solution. Think carefully about which parameter appears directly in the linear model and which requires a further calculation to recover.

7 Student C — Model Evaluation and Failure Prediction

Student C — `integration/integration.sci`

Your role in the system. Before the team can trust the calibration or degradation models built by Students A and B, someone must *measure* how good they are. That is your first task. Your second task — predicting the failure hour — is the central engineering output that the whole project is working toward. The maintenance team needs a credible,

numerically defensible answer to the question: "How many more hours does this pump have?"

Task C1: `goodness_of_fit(y_actual, y_pred)` — Model Accuracy Metrics

- **Input:** measured values `y_actual` ($n \times 1$), model predictions `y_pred` ($n \times 1$)
- **Output:** `rmse` (Root-Mean-Square Error, same units as y) and `r2` (coefficient of determination, dimensionless)

The formulas for both metrics are provided in Section 3 (Theoretical Background). Your task is to implement them correctly in Scilab and handle the edge cases described there.

Guiding questions:

- Describe in words what RMSE tells you that R^2 does not, and vice versa.
- What does it mean physically when a model achieves $R^2 = 0$? When it achieves $R^2 < 0$?
- Under what data condition does the R^2 formula become undefined? How should you handle this in code so the function never crashes?
- How would you quickly verify your implementation is correct without running the full project? Design a simple test case where you know the exact expected output.

Task C2: `find_threshold_hour(hours, vib_fit, threshold)` — Failure Hour Prediction

- **Input:** sorted hour vector `hours` (n), fitted vibration vector `vib_fit` (n, smooth and monotone increasing), `threshold` scalar
- **Output:** `h_star` — predicted failure hour; return `%inf` if the threshold is not reached within the data range

Guiding questions:

- The fitted vibration curve eventually crosses the failure threshold. How can you scan through the `vib_fit` vector to identify a pair of consecutive points that straddle the crossing? What mathematical property do you look for?
- Once you have an interval $[h_L, h_R]$ that brackets the crossing, what does the Intermediate Value Theorem tell you about the existence of a root within that interval?
- The bisection method repeatedly narrows the bracket. At each step, you evaluate g at the midpoint of the current bracket. Based on the sign of this value, what happens to the bracket?
- How small does the bracket need to be for the result to be useful as a maintenance prediction? How do you translate that into a stopping condition?
- Why should you use the smooth `vib_fit` rather than the raw `vibration` data for this calculation? What would happen if you used the noisy raw signal?
- What should your function return if the vibration never exceeds the threshold within the available hour range?

Expected Outcomes — How to Verify Your Work

- **Perfect-fit check:** calling `goodness_of_fit(y, y)` with any non-trivial vector `y` must return `rmse` $= 0$ and `r2` $= 1$.
- **Mean-prediction check:** if `y_pred` is a constant vector equal to the mean of `y_actual`, then R^2 must be exactly 0.
- **Model quality:** both the polynomial (Student A) and exponential (Student B) models should produce $R^2 > 0.95$ on the provided dataset if implemented correctly. A lower value indicates a bug in either this function or the upstream model.
- **Failure hour range:** the predicted `h_star` should be a physically meaningful positive number well beyond the end of the observed data range. A result of `%inf` or a negative number indicates a problem in the bracket scan or bisection logic.
- **Bracket sanity:** before running bisection, manually verify that the two bracket endpoints straddle the threshold (one fitted vibration below, one at or above the threshold).

Common Pitfalls

- Use the smooth fitted vibration curve from Student B, not the raw noisy measurements, to search for the threshold crossing.
- When scanning for the bracket, track your position by an index into the `hours` vector. Relying on floating-point equality tests to find a specific hour value later is fragile and error-prone.
- A finite number of bisection iterations (with a `break` condition) is safer than an unbounded `while` loop. Make sure your stopping condition is based on the width of the bracket, not on a fixed count.
- The R^2 denominator can be zero — guard against division by zero.

8 Student D — Data Loading and Health Dashboard

Student D — `load_data.sci` & `health_dashboard.sci`

Your role in the system. Student D owns both ends of the data pipeline: you are responsible for getting the raw sensor data into the system at the start (Task D1), computing the normalised health score from the results at the end (Task D2–D3), and orchestrating the entire pipeline in `main.sce` (Task D4). Your module is the glue that connects every other module into a working system.
Good data loading is not glamorous, but it is critical — a subtle parsing error here corrupts every downstream result silently. Similarly, the final dashboard is the only output that a maintenance engineer will actually look at, so it must be clear, complete, and correctly labelled.

Task D1: `load_data(fname)` (in `data_loader/load_data.sci`)
Read and parse the CSV sensor file and return four numeric column vectors.

- **Input:** `fname` — full path string to `pump_health_data.csv`
- **Output:** four column vectors `pressure`, `voltage`, `hours`, `vibration`, each of length $n = 120$

Guiding questions:

- Scilab's `read_csv(fname, ',')` returns a *matrix of strings* — all values, including numbers, come back as text. What sequence of steps do you need to perform to obtain a numeric matrix?
- The first line of the file is a header starting with `#` and must not be treated as data. How do you skip it?
- Once you have the numeric matrix, how do you extract each of the four columns as individual vectors?
- What should the function do if the specified file does not exist? How does Scilab let you check for file existence?

Expected Outcomes — Task D1

- All four output vectors must have exactly 120 elements.
- `hours` must be strictly increasing (monotone) from approximately 200 h to approximately 5000 h.
- `vibration` must contain only positive values (no zeros or negatives) — Student B depends on this.
- `pressure` values should be in the range 1–12 bar; `voltage` values should span approximately 1–8 mV.

Task D2: `compute_health_index(vibration, threshold)` (in `user_interface/health_dashboard.sci`)

- **Input:** `vibration` ($n \times 1$, raw or fitted), `threshold` scalar (failure vibration level)
- **Output:** `HI` — Health Index vector of the same length, with values clamped to $[0, 1]$

Guiding questions:

- Define HI so that it equals 0 when the pump is perfectly healthy and 1 when vibration has exactly reached the failure threshold. Write a one-line formula relating HI, ξ, and ξ_{th}.
- What happens to vibration beyond the threshold? Without clamping, what would HI become? Why must it be clamped to 1?
- Verify your formula gives sensible values at the two endpoints: $\xi = 0$ and $\xi = \xi_{\text{th}}$.

Task D3: `plot_report(hours, vibration, vib_fit, HI, threshold, h_star)` (in `user_interface/health_dashboard.sci`)

Generate a professional two-panel figure and save it as `pump_health_report_v2.png`. Figure 1 shows the **exact target output** your function must reproduce.

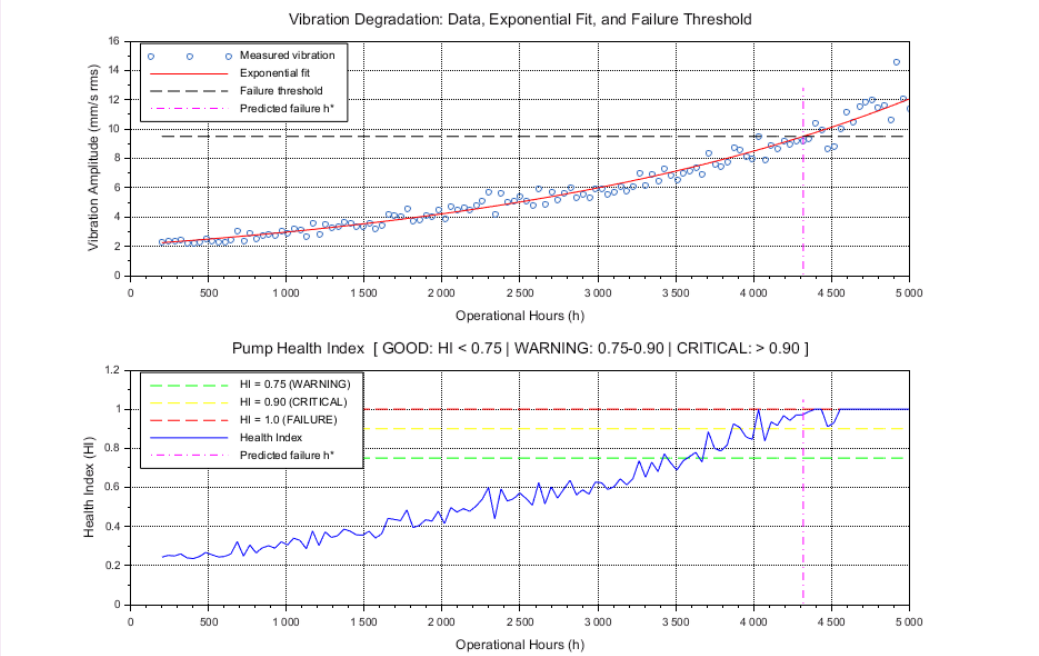

Figure 1: **Target dashboard output** (`pump_health_report_v2.png`). Panel 1 shows the measured vibration data (blue circles), the exponential fitted curve (red solid), the failure threshold at 9.5 mm/s_{rms} (black dashed), and the predicted failure marker h^* (magenta dash-dot). Panel 2 shows the Health Index (blue solid) together with the GOOD/WARNING boundary at HI = 0.75 (green dashed), the WARNING/CRITICAL boundary at HI = 0.90 (yellow dashed), the failure level at HI = 1.0 (red dashed), and the h^* marker (magenta dash-dot).

The required plot elements and their Scilab style codes are fixed — follow them exactly (the automated tests check the output file name and that the function runs without error):

Panel	Series	Style
1	Measured vibration (scatter)	`scatter(..., 18, [0.15 0.40 0.75])`
1	Exponential fit	`'r-'`
1	Failure threshold (9.5)	`'k--'`
1	Failure marker h^*	`'m-.'`, $y \in [0,\ 1.35\,\xi_{\text{th}}]$
2	HI = 0.75 (WARNING boundary)	`'g--'`
2	HI = 0.90 (CRITICAL boundary)	`'y--'`
2	HI = 1.0 (FAILURE level)	`'r--'`
2	Health Index curve	`'b-'`
2	Failure marker h^*	`'m-.'`, $y \in [0,\ 1.05]$

Guiding questions:

- A horizontal line at constant value c must be drawn as a plot of two vectors of matching length — not as a scalar. How do you construct the y-data vector for such a line?
- What Scilab function creates a new figure window? How do you clear any previous plot before drawing? What command divides the figure into two sub-panels?
- How do you save the figure as a PNG file?
- The failure marker at h^* should only appear when a finite failure hour was predicted. How do you make this conditional?

- Every plotted series needs a legend entry. What goes wrong if the number of legend labels does not exactly match the number of plotted series?

Task D4: Complete `main.sce` — orchestrate the full pipeline by calling each module's functions in the correct order and printing a diagnostic summary to the console.

- Call `load_data` to obtain the four sensor vectors.
- Call Student A's functions to build and evaluate the pressure calibration model.
- Call Student B's functions to build and evaluate the vibration degradation model.
- Call Student C's functions to compute goodness-of-fit metrics and predict the failure hour.
- Call Student D's own functions to compute the Health Index and generate the dashboard figure.
- Print a readable summary to the Scilab console that includes the number of rows loaded, the model parameters, the accuracy metrics for both models, and the predicted failure hour.

The order in which you call these functions is not arbitrary — some outputs are required as inputs by later stages. Map out the data dependencies before you start writing code.

Expected Outcomes — Tasks D2–D4

- At the beginning of the dataset (low vibration) HI should be well below 1.0; near the end it should approach or reach 1.0 (see Figure 1).
- Your saved `pump_health_report_v2.png` must visually match Figure 1: upward vibration trend in Panel 1 with the exponential fit following the scatter, the 9.5 mm/s threshold line, and all legend entries present. Panel 2 must show the HI rising from ≈ 0.25 to 1.0 with all three zone boundary lines visible.
- Check that the PNG file is created after running `main.sce`.
- The console output should clearly report: data row count (120), exponential model parameters (`a` and `b`), RMSE and R^2 for both models, and the predicted failure hour (≈ 4319 h).

Common Pitfalls

- To draw a horizontal line across the whole plot at height c, you need a vector of c repeated n times — Scilab will not accept a scalar where a vector is required.
- Plot the failure hour marker conditionally. Adding a marker at an infinite or undefined position will either error or produce a misleading figure.
- The number of entries in your legend call must match the number of plotted series in that panel exactly. Count them before finalising.
- In `main.sce`, confirm the execution order respects the dependency chain: you cannot compute HI before you have the vibration model, and you cannot print the failure hour before Student C's function has been called.

9 Running and Testing Your Code

9.1 Running the Main Script

Open Scilab, change to the `phm_project/` directory, and run:

```
exec('main.sce', -1);
```

When all four modules are correctly implemented, the script produces two output figures. Figure 2 shows the **4-panel diagnostic overview** (saved as `pump_health_diagnostic_v2.png`) that summarises the outputs of every student module in a single view.

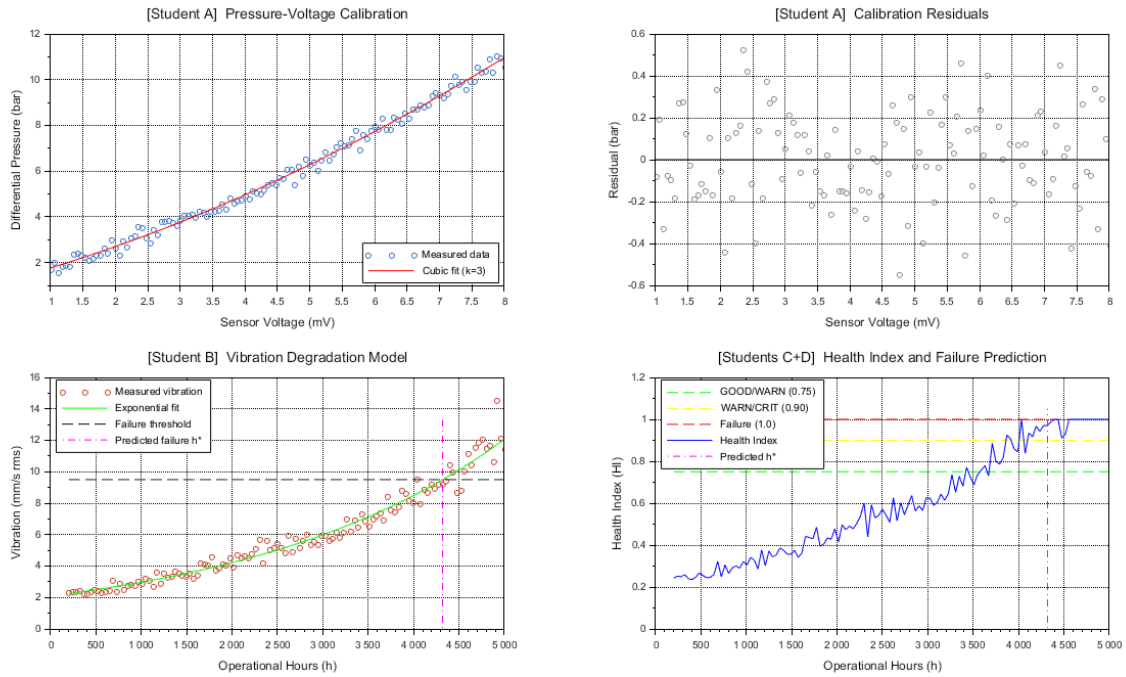

Figure 2: **Expected 4-panel diagnostic output** (`pump_health_diagnostic_v2.png`) produced by `main_solution.sce` with all modules correctly implemented. **Top-left** [Student A]: pressure vs. voltage scatter with the fitted cubic calibration curve. **Top-right** [Student A]: calibration residuals — should be randomly scattered around zero with no systematic trend. **Bottom-left** [Student B]: vibration data (orange circles) with the exponential fit (green), failure threshold (black dashed), and predicted failure marker h^* (magenta). **Bottom-right** [Students C+D]: Health Index over time with the GOOD/WARNING/CRITICAL zone boundary lines and h^* marker. Compare your integrated output with this figure before submission.

9.2 Running the Tests

```
exec('test_suites/run_all_tests.sci', -1);
```

Each test reports `PASS` or `FAIL`. Aim for all tests to pass before submission.

9.3 Debugging Tips

Common Issues and How to Resolve Them

- **Dimension mismatch ("inconsistent row/column dimensions"):** All arithmetic on vectors must use matching orientations. As a habit, force column vectors at the top of every function: `x = x(:); y = y(:);`.
- **`read_csv` error or empty data:** Verify the full path with `disp(data_file)` before calling `load_data`. Use `isfile(fname)` to check existence.
- **Singular or near-singular system:** The design matrix **Z** becomes poorly conditioned when k is too large relative to the data range. For this project use exactly $k = 3$.
- **`NaN` or `-Inf` from `log`:** `log` requires strictly positive input. Print `min(vibration)`

to verify there are no zero or negative readings before calling `linearize_vib`.

- `h_star = %inf`: This means the bisection scan found no sign change — the fitted vibration curve never exceeds the threshold in the data range. Check `max(vib_fit)` vs. `THRESHOLD` to diagnose.
- **Legend count error:** The number of labels in your `legend([...], pos)` call must match the number of plot/scatter series drawn in that panel exactly.
- **Undefined variable `any`:** Scilab does not have a built-in `any()` function in all versions. Replace `any(x < 0)` with `sum(x < 0) > 0`.

10 Submission Checklist

Submit **your individual file(s) only**:

Student	File(s) to submit	What will be tested
A	`StudentA/interpolation.sci`	`poly_fit` and `eval_poly`: 5 unit tests + integration test
B	`StudentB/differentiation.sci`	`linearize_vib` and `predict_vibration`: 5 unit tests + integration test
C	`StudentC/integration.sci`	`goodness_of_fit` and `find_threshold_hour`: 5 unit tests + integration test
D	`StudentD/load_data.sci`, `StudentD/health_dashboard.sci`, `StudentD/main.sce`	`load_data`, `compute_health_index`, `plot_report`, and `main.sce` integration run

Pre-Submission Checklist

- ☐ All functions in your file run without errors.
- ☐ `exec('test_suites/run_all_tests.sci', -1)` shows all `PASS`.
- ☐ Your function signatures exactly match those in the skeleton (function name, number and order of inputs/outputs).
- ☐ No calls to forbidden functions (`polyfit`, `lsq`, `inv`, `reglin`).
- ☐ Student D: `pump_health_report_v2.png` is generated.
- ☐ All variable names and comments are in English.

Academic Integrity

All code must be your own work. Each function is tested individually and cross-checked. Identical implementations between different students will be investigated under the academic integrity policy. You must be able to explain every line of your code during a viva if asked.

A Scilab Quick Reference

This appendix lists every Scilab built-in function used in this project. It is not a complete language manual — for full documentation, open Scilab and type `help function_name` in the console, or visit `https://help.scilab.org`.

A.1 Environment and File I/O

Scilab call	What it does
`exec(fname, -1)`	Run a script or load a `.sci` file silently.
`disp(x)`	Print the value of `x` to the console.
`mprintf('fmt', v)`	Formatted output (like C's `printf`).
`isfile(fname)`	Returns `%t` if `fname` exists; `%f` otherwise.
`read_csv(fname, ',')`	Read a CSV file; returns a *matrix of strings*.
`evstr(S)`	Convert string matrix `S` to a numeric matrix.
`fullfile(a, b)`	Join path components with the OS directory separator.
`error(msg)`	Stop execution and print an error message.

A.2 Vectors and Matrices

Scilab call	What it does
`v = [1; 2; 3]`	Column vector — semicolons separate rows.
`v = [1, 2, 3]`	Row vector — commas or spaces separate columns.
`v = v(:)`	Force `v` to a column vector. **Use at the top of every function.**
`n = length(v)`	Number of elements in vector `v`.
`[m,n] = size(A)`	Dimensions: `m` rows, `n` columns.
`zeros(m, n)`	$m \times n$ matrix of zeros (pre-allocate output).
`ones(m, n)`	$m \times n$ matrix of ones.
`linspace(a, b, n)`	`n` equally-spaced values from `a` to `b`.
`v(2:$)`	Elements from index 2 to the last (`$` = last index).
`A'`	Transpose of matrix `A`.
`A \ b`	Solve $\mathbf{Ax} = \mathbf{b}$; gives least-squares solution when `A` has more rows than columns.

A.3 Arithmetic, Math Functions, and Logical Tests

Scilab call	What it does
`a .* b`	Element-wise multiply; both operands must be the same size.
`a ./ b`	Element-wise divide.
`a .^ k`	Raise each element of `a` to the power `k`.
`log(x)`	Natural logarithm; `x` must be > 0 for all elements.
`exp(x)`	Exponential e^x, element-wise.
`sqrt(x)`	Square root, element-wise.
`abs(x)`	Absolute value, element-wise.
`mean(x)`	Arithmetic mean of all elements.
`sum(x)`	Sum of all elements.
`min(x) / max(x)`	Minimum / maximum element.
`isinf(x)`	`%t` if `x` equals `%inf` or `-%inf`.
`isnan(x)`	`%t` if `x` is `NaN` (not-a-number).
`sum(x < 0) > 0`	Test whether *any* element of `x` is negative (`any()` may not be available in all Scilab versions).

A.4 Control Flow

```
// For loop (ascending):
for i = 1:n
```

```
    // body
end

// For loop (descending --- used in Horner's method):
for j = length(coeff)-1 : -1 : 1
    result = coeff(j) + x_val * result;
end

// If / elseif / else:
if condition
    // ...
elseif other_condition
    // ...
else
    // ...
end

// Break out of a loop early (used in bisection bracket scan):
for i = 1:n-1
    if (f(i)) * (f(i+1)) <= 0
        i_lo = i;
        break;
    end
end
```

A.5 Plotting

Scilab call	What it does
`fig = scf(n)`	Open / activate figure window number `n`.
`clf()`	Clear the current figure window.
`subplot(r, c, k)`	Activate sub-panel `k` in an $r \times c$ grid.
`plot(x, y, 'b-')`	Line plot. Style codes: `'b-'` solid blue, `'r--'` dashed red, `'k-.'` dash-dot black.
`scatter(x, y, sz, rgb)`	Scatter plot. `sz` = marker size (e.g. 18); `rgb` = colour as a $[r\ g\ b]$ row vector in $[0, 1]$.
`xlabel('text')`	Label the x-axis.
`ylabel('text')`	Label the y-axis.
`title('text')`	Panel title.
`legend([s1;s2;s3], pos)`	Legend; `pos` 1–4 (corners) or 5 (no box). Number of strings must equal number of plotted series.
`xgrid()`	Draw a light grid on the current axes.
`xs2png(fig, fname)`	Save figure `fig` as a PNG file `fname`.

A.6 Complete Scilab Idioms for This Project

The following code snippets cover every common pattern you will need. Copy, adapt, and combine them — but make sure you understand *why* each line works before using it.

```
// ---- Force column vectors (top of every function) ----
x = x(:);   y = y(:);   n = length(x);

// ---- Build degree-k design matrix ----
Z = zeros(n, k+1);
for j = 1:k+1
    Z(:, j) = x .^ (j-1);
end

```

```
// ---- Solve Normal Equations (least-squares fit) ----
c = (Z' * Z) \ (Z' * y);    // c = [c0; c1; ...; ck]

// ---- Horner's method (evaluate polynomial at one point x_val) ----
result = coeff(length(coeff));           // start at highest-degree coeff
for j = length(coeff)-1 : -1 : 1
    result = coeff(j) + x_val * result;  // accumulate from inside out
end

// ---- Check file exists before reading ----
if ~isfile(fname)
    error('File not found: ' + fname);
end

// ---- Read CSV, skip header row, convert to numbers ----
raw  = read_csv(fname, ',');
data = evstr(raw(2:$, :));   // row 1 is the header line

// ---- Draw a horizontal line at height c across n points ----
plot(hours, c * ones(n,1), 'k--');

// ---- Scatter with a custom RGB colour ----
scatter(hours, vibration, 18, [0.15 0.40 0.75]);

// ---- Conditional legend (add h* marker only when it is finite) ----
if ~isinf(h_star)
    plot([h_star, h_star], [0, y_max], 'm-.');
    legend(['data', 'fit', 'threshold', 'h*'], 4);
else
    legend(['data', 'fit', 'threshold'], 4);
end

// ---- Formatted console output ----
mprintf('  Rows loaded      : %d\n',     n);
mprintf('  Poly fit R2      : %.4f\n',   r2_A);
mprintf('  Failure at h*    : %.1f h\n', h_star);
```